

SUGGESTED SKILLS

1.B

Determine what knowledge can be extracted from data.

5.A

Explain how computing impacts society, economy, and culture.

TOPIC 4.1

Ethical and Social Issues Around Data Collection

Required Course Content

LEARNING OBJECTIVE

4.1.A

Explain the risks to privacy from collecting and storing personal data on computer systems.

4.1.B

Explain the importance of recognizing data quality and potential issues when using a data set.

4.1.C

Identify an appropriate data set to use in order to solve a problem or answer a specific question.

ESSENTIAL KNOWLEDGE

4.1.A.1

When using a computer, personal privacy is at risk. When developing new programs, programmers should attempt to safeguard the personal privacy of the user.

4.1.B.1

Algorithmic bias describes systemic and repeated errors in a program that create unfair outcomes for a specific group of users.

4.1.B.2

Programmers should be aware of the data set collection method and the potential for bias when using this method before using the data to extrapolate new information or drawing conclusions.

4.1.B.3

Some data sets are incomplete or contain inaccurate data. Using such data in the development or use of a program can cause the program to work incorrectly or inefficiently.

4.1.C.1

Contents of a data set might be related to a specific question or topic and might not be appropriate to give correct answers or extrapolate information for a different question or topic.

TOPIC 4.2

Introduction to Using Data Sets

SUGGESTED SKILLS

1.B

Determine what knowledge can be extracted from data.

Required Course Content

LEARNING OBJECTIVE**4.2.A**

Represent patterns and algorithms that involve data sets found in everyday life using written language or diagrams.

ESSENTIAL KNOWLEDGE**4.2.A.1**

A *data set* is a collection of specific pieces of information or data.

4.2.A.2

Data sets can be manipulated and analyzed to solve a problem or answer a question. When analyzing data sets, values within the set are accessed and utilized one at a time and then processed according to the desired outcome.

4.2.A.3

Data can be represented in a diagram by using a chart or table. This visual can be used to plan the algorithm that will be used to manipulate the data.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

TOPIC 4.3

Array Creation and Access

Required Course Content

LEARNING OBJECTIVE

4.3.A

Develop code used to represent collections of related data using one-dimensional (1D) array objects.

ESSENTIAL KNOWLEDGE

4.3.A.1

An *array* stores multiple values of the same type. The values can be either primitive values or object references.

4.3.A.2

The length of an array is established at the time of creation and cannot be changed. The length of an array can be accessed through the `length` attribute.

4.3.A.3

When an array is created using the keyword `new`, all of its elements are initialized to the default values for the element data type. The default value for `int` is `0`, for `double` is `0.0`, for `boolean` is `false`, and for a reference type is `null`.

4.3.A.4

Initializer lists can be used to create and initialize arrays.

4.3.A.5

Square brackets `[]` are used to access and modify an element in a 1D array using an index.

4.3.A.6

The valid index values for an array are `0` through one less than the length of the array, inclusive. Using an index value outside of this range will result in an `ArrayIndexOutOfBoundsException`.

TOPIC 4.4

Array Traversals

SUGGESTED SKILLS**2.B**

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

Required Course Content

LEARNING OBJECTIVE**4.4.A**

Develop code used to traverse the elements in a 1D array and determine the result of these traversals.

ESSENTIAL KNOWLEDGE**4.4.A.1**

Traversing an array is when repetition statements are used to access all or an ordered sequence of elements in an array.

4.4.A.2

Traversing an array with an indexed `for` loop or `while` loop requires elements to be accessed using their indices.

4.4.A.3

An enhanced `for` loop header includes a variable, referred to as the enhanced `for` loop variable. For each iteration of the enhanced `for` loop, the enhanced `for` loop variable is assigned a copy of an element without using its index.

4.4.A.4

Assigning a new value to the enhanced `for` loop variable does not change the value stored in the array.

4.4.A.5

When an array stores object references, the attributes can be modified by calling methods on the enhanced `for` loop variable. This does not change the object references stored in the array.

4.4.A.6

Code written using an enhanced `for` loop to traverse elements in an array can be rewritten using an indexed `for` loop or a `while` loop.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

4.A

Describe the behavior of a code segment or program.

TOPIC 4.5

Implementing Array Algorithms

Required Course Content

LEARNING OBJECTIVE

4.5.A

Develop code for standard and original algorithms for a particular context or specification that involves arrays and determine the result of these algorithms.

ESSENTIAL KNOWLEDGE

4.5.A.1

There are standard algorithms that utilize array traversals to:

- determine a minimum or maximum value
- compute a sum or average
- determine if at least one element has a particular property
- determine if all elements have a particular property
- determine the number of elements having a particular property
- access all consecutive pairs of elements
- determine the presence or absence of duplicate elements
- shift or rotate elements left or right
- reverse the order of the elements

TOPIC 4.6

Using Text Files

SUGGESTED SKILLS

2.C

Write program code involving procedural abstractions.

3.C

Determine the result or output based on code that contains procedural abstractions.

4.B

Describe the initial conditions that must be met for a code segment to work as intended or described.

Required Course Content

LEARNING OBJECTIVE

4.6.A

Develop code to read data from a text file.

ESSENTIAL KNOWLEDGE

4.6.A.1

A *file* is storage for data that persists when the program is not running. The data in a file can be retrieved during program execution.

4.6.A.2

A file can be connected to the program using the `File` and `Scanner` classes.

4.6.A.3

A file can be opened by creating a `File` object, using the name of the file as the argument of the constructor.

- `File(String str)` is the `File` constructor that accepts a `String` file name to open for reading, where `str` is the pathname for the file.

4.6.A.4

When using the `File` class, it is required to indicate what to do if the file with the provided name cannot be opened. One way to accomplish this is to add `throws IOException` to the header of the method that uses the file. If the file name is invalid, the program will terminate.

4.6.A.5

The `File` and `IOException` classes are part of the `java.io` package. An `import` statement must be used to make these classes available for use in the program.

continued on next page

LEARNING OBJECTIVE

4.6.A

Develop code to read data from a text file.

ESSENTIAL KNOWLEDGE

4.6.A.6

The following `Scanner` methods and constructor—including what they do and when they are used—are part of the Java Quick Reference:

- `Scanner(File f)` is the `Scanner` constructor that accepts a `File` for reading.
- `int nextInt()` returns the next `int` read from the file or input source if available. If the next `int` does not exist or is out of range, it will result in an `InputMismatchException`.
- `double nextDouble()` returns the next `double` read from the file or input source. If the next `double` does not exist, it will result in an `InputMismatchException`.
- `boolean nextBoolean()` returns the next `boolean` read from the file or input source. If the next `boolean` does not exist, it will result in an `InputMismatchException`.
- `String nextLine()` returns the next line of text as a `String` read from the file or input source; can return the empty string if called immediately after another `Scanner` method that is reading from the file or input source.
- `String next()` returns the next `String` read from the file or input source.
- `boolean hasNext()` returns `true` if there is a next item to read in the file or input source; returns `false` otherwise.
- `void close()` closes this scanner.

✖ EXCLUSION STATEMENT—Accepting input from the keyboard is outside the scope of the AP Computer Science A course and exam.

4.6.A.7

Using `nextLine` and the other `Scanner` methods together on the same input source sometimes requires code to adjust for the methods' different ways of handling whitespace.

✖ EXCLUSION STATEMENT—Writing or analyzing code that uses both `nextLine` and other `Scanner` methods on the same input source is outside the scope of the AP Computer Science A course and exam.

continued on next page

LEARNING OBJECTIVE

4.6.A

Develop code to read data from a text file.

ESSENTIAL KNOWLEDGE

4.6.A.8

The following additional `String` method—including what it does and when it is used—is part of the Java Quick Reference:

- `String[] split(String del)` returns a `String` array where each element is a substring of `this String`, which has been split around matches of the given expression `del`.

EXCLUSION STATEMENT—*The parameter `del` uses a format called a regular expression. Writing or analyzing code that uses any of the special properties of regular expressions (e.g., `\\*`, `\\.`) is outside the scope of the AP Computer Science A course and exam.*

4.6.A.9

A `while` loop can be used to detect if the file still contains elements to read by using the `hasNext` method as the condition of the loop.

4.6.A.10

A file should be closed when the program is finished using it. The `close` method from `Scanner` is called to close the file.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

TOPIC 4.7

Wrapper Classes

Required Course Content

LEARNING OBJECTIVE

4.7.A

Develop code to use `Integer` and `Double` objects from their primitive counterparts and determine the result of using these objects.

ESSENTIAL KNOWLEDGE

4.7.A.1

The `Integer` class and `Double` class are part of the `java.lang` package. An `Integer` object is immutable, meaning once an `Integer` object is created, its attributes cannot be changed. A `Double` object is immutable, meaning once a `Double` object is created, its attributes cannot be changed.

4.7.A.2

Autoboxing is the automatic conversion that the Java compiler makes between primitive types and their corresponding object wrapper classes. This includes converting an `int` to an `Integer` and a `double` to a `Double`. The Java compiler applies autoboxing when a primitive value is:

- passed as a parameter to a method that expects an object of the corresponding wrapper class
- assigned to a variable of the corresponding wrapper class

4.7.A.3

Unboxing is the automatic conversion that the Java compiler makes from the wrapper class to the primitive type. This includes converting an `Integer` to an `int` and a `Double` to a `double`. The Java compiler applies unboxing when a wrapper class object is:

- passed as a parameter to a method that expects a value of the corresponding primitive type
- assigned to a variable of the corresponding primitive type

continued on next page

LEARNING OBJECTIVE

4.7.A

Develop code to use `Integer` and `Double` objects from their primitive counterparts and determine the result of using these objects.

ESSENTIAL KNOWLEDGE

4.7.A.4

The following class `Integer` method—including what it does and when it is used—is part of the Java Quick Reference:

- `static int parseInt(String s)` returns the `String` argument as an `int`.

4.7.A.5

The following class `Double` method—including what it does and when it is used—is part of the Java Quick Reference:

- `static double parseDouble(String s)` returns the `String` argument as a `double`.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

TOPIC 4.8

ArrayList Methods

Required Course Content

LEARNING OBJECTIVE

4.8.A

Develop code for collections of related objects using `ArrayList` objects and determine the result of calling methods on these objects.

ESSENTIAL KNOWLEDGE

4.8.A.1

An `ArrayList` object is mutable in size and contains object references.

4.8.A.2

The `ArrayList` constructor `ArrayList()` constructs an empty list.

4.8.A.3

Java allows the generic type `ArrayList<E>`, where the type parameter `E` specifies the type of the elements. When `ArrayList<E>` is specified, the types of the reference parameters and return type when using the `ArrayList` methods are type `E`. `ArrayList<E>` is preferred over `ArrayList`. For example, `ArrayList<String> names = new ArrayList<String>();` allows the compiler to find errors that would otherwise be found at run-time.

4.8.A.4

The `ArrayList` class is part of the `java.util` package. An `import` statement must be used to make this class available for use in the program.

continued on next page

LEARNING OBJECTIVE

4.8.A

Develop code for collections of related objects using `ArrayList` objects and determine the result of calling methods on these objects.

ESSENTIAL KNOWLEDGE

4.8.A.5

The following `ArrayList` methods—including what they do and when they are used—are part of the Java Quick Reference:

- `int size()` returns the number of elements in the list.
- `boolean add(E obj)` appends `obj` to end of list; returns `true`.
- `void add(int index, E obj)` inserts `obj` at position `index` ($0 \leq \text{index} \leq \text{size}$), moving elements at position `index` and higher to the right (adds 1 to their indices) and adds 1 to size.
- `E get(int index)` returns the element at position `index` in the list.
- `E set(int index, E obj)` replaces the element at position `index` with `obj`; returns the element formerly at position `index`.
- `E remove(int index)` removes element from position `index`, moving elements at position `index + 1` and higher to the left (subtracts 1 from their indices) and subtracts 1 from size; returns the element formerly at position `index`.

4.8.A.6

The indices for an `ArrayList` start at 0 and end at the number of elements $- 1$.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

4.A

Describe the behavior of a code segment or program.

TOPIC 4.9

ArrayList Traversals

Required Course Content

LEARNING OBJECTIVE

4.9.A

Develop code used to traverse the elements of an `ArrayList` and determine the results of these traversals.

ESSENTIAL KNOWLEDGE

4.9.A.1

Traversing an `ArrayList` is when iteration or recursive statements are used to access all or an ordered sequence of the elements in an `ArrayList`.

4.9.A.2

Deleting elements during a traversal of an `ArrayList` requires the use of special techniques to avoid skipping elements.

4.9.A.3

Attempting to access an index value outside of its range will result in an `IndexOutOfBoundsException`.

4.9.A.4

Changing the size of an `ArrayList` while traversing it using an enhanced `for` loop can result in a `ConcurrentModificationException`. Therefore, when using an enhanced `for` loop to traverse an `ArrayList`, you should not add or remove elements.

TOPIC 4.10

Implementing ArrayList Algorithms

Required Course Content

LEARNING OBJECTIVE

4.10.A

Develop code for standard and original algorithms for a particular context or specification that involve `ArrayList` objects and determine the result of these algorithms.

ESSENTIAL KNOWLEDGE

4.10.A.1

There are standard `ArrayList` algorithms that utilize traversals to:

- determine a minimum or maximum value
- compute a sum or average
- determine if at least one element has a particular property
- determine if all elements have a particular property
- determine the number of elements having a particular property
- access all consecutive pairs of elements
- determine the presence or absence of duplicate elements
- shift or rotate elements left or right
- reverse the order of the elements
- insert elements
- delete elements

4.10.A.2

Some algorithms require multiple `String`, array, or `ArrayList` objects to be traversed simultaneously.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

4.A

Describe the behavior of a code segment or program.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

TOPIC 4.11

2D Array Creation and Access

Required Course Content

LEARNING OBJECTIVE

4.11.A

Develop code used to represent collections of related data using two-dimensional (2D) array objects.

ESSENTIAL KNOWLEDGE

4.11.A.1

A 2D array is stored as an array of arrays. Therefore, the way 2D arrays are created and indexed is similar to 1D array objects. The size of a 2D array is established at the time of creation and cannot be changed. 2D arrays can store either primitive data or object reference data.

EXCLUSION STATEMENT—*Nonrectangular 2D array objects are outside the scope of the AP Computer Science A course and exam.*

4.11.A.2

When a 2D array is created using the keyword `new`, all of its elements are initialized to the default values for the element data type. The default value for `int` is 0, for `double` is 0.0, for `boolean` is `false`, and for a reference type is `null`.

4.11.A.3

The initializer list used to create and initialize a 2D array consists of initializer lists that represent 1D arrays; for example, `int[][] arr2D = { {1, 2, 3}, {4, 5, 6} };`.

4.11.A.4

The square brackets `[row][col]` are used to access and modify an element in a 2D array. For the purposes of the exam, when accessing the element at `arr[first][second]`, the first index is used for rows, the second index is used for columns.

continued on next page

LEARNING OBJECTIVE

4.11.A

Develop code used to represent collections of related data using two-dimensional (2D) array objects.

ESSENTIAL KNOWLEDGE

4.11.A.5

A single array that is a row of a 2D array can be accessed using the 2D array name and a single set of square brackets containing the row index.

4.11.A.6

The number of rows contained in a 2D array can be accessed through the `length` attribute. The valid row index values for a 2D array are 0 through one less than the number of rows or the length of the array, inclusive. The number of columns contained in a 2D array can be accessed through the `length` attribute of one of the rows. The valid column index values for a 2D array are 0 through one less than the number of columns or the length of any given row of the array, inclusive. For example, given a 2D array named `values`, the number of rows is `values.length` and the number of columns is `values[0].length`. Using an index value outside of these ranges will result in an `ArrayIndexOutOfBoundsException`.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

TOPIC 4.12

2D Array Traversals

Required Course Content

LEARNING OBJECTIVE

4.12.A

Develop code used to traverse the elements in a 2D array and determine the result of these traversals.

ESSENTIAL KNOWLEDGE

4.12.A.1

Nested iteration statements are used to traverse and access all or an ordered sequence of elements in a 2D array. Since 2D arrays are stored as arrays of arrays, the way 2D arrays are traversed using `for` loops and enhanced `for` loops is similar to 1D array objects. Nested iteration statements can be written to traverse the 2D array in row-major order, column-major order, or a uniquely defined order. *Row-major order* refers to an ordering of 2D array elements where traversal occurs across each row, whereas *column-major order* traversal occurs down each column.

4.12.A.2

The outer loop of a nested enhanced `for` loop used to traverse a 2D array traverses the rows. Therefore, the enhanced `for` loop variable must be the type of each row, which is a 1D array. The inner loop traverses a single row. Therefore, the inner enhanced `for` loop variable must be the same type as the elements stored in the 1D array. Assigning a new value to the enhanced `for` loop variable does not change the value stored in the array.

TOPIC 4.13

Implementing 2D Array Algorithms

Required Course Content

LEARNING OBJECTIVE

4.13.A

Develop code for standard and original algorithms for a particular context or specification that involves 2D arrays and determine the result of these algorithms.

ESSENTIAL KNOWLEDGE

4.13.A.1

There are standard algorithms that utilize 2D array traversals to:

- determine a minimum or maximum value of all the elements or for a designated row, column, or other subsection
- compute a sum or average of all the elements or for a designated row, column, or other subsection
- determine if at least one element has a particular property in the entire 2D array or for a designated row, column, or other subsection
- determine if all elements of the 2D array or a designated row, column, or other subsection have a particular property
- determine the number of elements in the 2D array or in a designated row, column, or other subsection having a particular property
- access all consecutive pairs of elements
- determine the presence or absence of duplicate elements in the 2D array or in a designated row, column, or other subsection
- shift or rotate elements in a row left or right or in a column up or down
- reverse the order of the elements in a row or column

SUGGESTED SKILLS

2B

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

4.A

Describe the behavior of a code segment or program.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

4.A

Describe the behavior of a code segment or program.

TOPIC 4.14

Searching Algorithms

Required Course Content

LEARNING OBJECTIVE**4.14.A**

Develop code used for linear search algorithms to search for specific information in a collection and determine the results of executing a search.

ESSENTIAL KNOWLEDGE**4.14.A.1**

Linear search algorithms are standard algorithms that check each element in order until the desired value is found or all elements in the array or `ArrayList` have been checked. Linear search algorithms can begin the search process from either end of the array or `ArrayList`.

4.14.A.2

When applying linear search algorithms to 2D arrays, each row must be accessed then linear search applied to each row of the 2D array.

TOPIC 4.15

Sorting Algorithms

SUGGESTED SKILLS

3.B

Determine the result or output based on code that contains data abstractions.

4.A

Describe the behavior of a code segment or program.

4.B

Describe the initial conditions that must be met for a code segment to work as intended or described.

Required Course Content

LEARNING OBJECTIVE

4.15.A

Determine the result of executing each step of sorting algorithms to sort the elements of a collection.

ESSENTIAL KNOWLEDGE

4.15.A.1

Selection sort and insertion sort are iterative sorting algorithms that can be used to sort elements in an array or `ArrayList`.

4.15.A.2

Selection sort repeatedly selects the smallest (or largest) element from the unsorted portion of the list and swaps it into its correct (and final) position in the sorted portion of the list.

4.15.A.3

Insertion sort inserts an element from the unsorted portion of a list into its correct (but not necessarily final) position in the sorted portion of the list by shifting elements of the sorted portion to make room for the new element.

SUGGESTED SKILLS

3.C

Determine the result or output based on code that contains procedural abstractions.

4.A

Describe the behavior of a code segment or program.

4.B

Describe the initial conditions that must be met for a code segment to work as intended or described.

TOPIC 4.16

Recursion

Required Course Content

LEARNING OBJECTIVE

4.16.A

Determine the result of calling recursive methods.

ESSENTIAL KNOWLEDGE

4.16.A.1

A *recursive method* is a method that calls itself. Recursive methods contain at least one base case, which halts the recursion, and at least one recursive call. Recursion is another form of repetition.

4.16.A.2

Each recursive call has its own set of local variables, including the parameters. Parameter values capture the progress of a recursive process, much like loop control variable values capture the progress of a loop.

4.16.A.3

Any recursive solution can be replicated through the use of an iterative approach and vice versa.

✖ EXCLUSION STATEMENT—*Writing recursive code is outside the scope of the AP Computer Science A course and exam.*

TOPIC 4.17

Recursive Searching and Sorting

SUGGESTED SKILLS

4.A

Describe the behavior of a code segment or program.

4.B

Describe the initial conditions that must be met for a code segment to work as intended or described.

Required Course Content

LEARNING OBJECTIVE

4.17.A

Determine the result of executing recursive algorithms that use strings or collections.

4.17.B

Determine the result of each iteration of a binary search algorithm used to search for information in a collection.

4.17.C

Determine the result of each iteration of the merge sort algorithm when used to sort a collection.

ESSENTIAL KNOWLEDGE

4.17.A.1

Recursion can be used to traverse `String` objects, arrays, and `ArrayList` objects.

4.17.B.1

Data must be in sorted order to use the binary search algorithm. *Binary search* starts at the middle of a sorted array or `ArrayList` and eliminates half of the array or `ArrayList` in each recursive call until the desired value is found or all elements have been eliminated.

4.17.B.2

Binary search is typically more efficient than linear search.

EXCLUSION STATEMENT—*Search algorithms other than linear and binary search are outside the scope of the AP Computer Science A course and exam.*

4.17.B.3

The binary search algorithm can be written either iteratively or recursively.

4.17.C.1

Merge sort is a recursive sorting algorithm that can be used to sort elements in an array or `ArrayList`.

EXCLUSION STATEMENT—*Sorting algorithms other than selection, insertion, and merge sort are outside the scope of the AP Computer Science A course and exam.*

continued on next page

LEARNING OBJECTIVE

4.17.C

Determine the result of each iteration of the merge sort algorithm when used to sort a collection.

ESSENTIAL KNOWLEDGE

4.17.C.2

Merge sort repeatedly divides an array into smaller subarrays until each subarray is one element and then recursively merges the sorted subarrays back together in sorted order to form the final sorted array.